

Setup Models

```
In [9]: from llm_provider_setup import build_chat_model

# Codespaces default: use "gemini" or "openai"
PROVIDER = "ollama"
API_KEY = None # real key for gemini/openai; use None for ollama
MODEL_NAME = "llama3.1:8b" # optional model override
# Ollama is local-only. Use this when running notebook locally on your laptop
OLLAMA_BASE_URL = "http://localhost:11434"

model, provider_name, resolved_model = build_chat_model(
    provider=PROVIDER,
    api_key=API_KEY,
    model_name=MODEL_NAME, # optional model override
    ollama_base_url=OLLAMA_BASE_URL,
)

print(f"Provider: {provider_name} | Model: {resolved_model}")
```

Provider: ollama | Model: llama3.1:8b

04.03. Setup tools for custom Agent

```
In [10]: import pandas as pd

#Load the golf orders CSV into a Pandas dataframe.
product_orders_df = pd.read_csv("data/golf_orders.csv")
print(product_orders_df)
```

	order_id	customer_name	email	
0	G1001	Alex Kim	alex@example.com	
1	G1002	Jordan Lee	jordan@example.com	
2	G1003	Taylor Smith	taylor@example.com	
3	G1004	Morgan Diaz	morgan@example.com	
4	G1005	Casey Brown	casey@example.com	

	product_name	category	status	order_date	
0	StormDrive 10.5° Driver	driver	Shipped	2026-03-15	
1	FairwayPro Iron Set (4-PW)	irons	Processing	2026-03-18	
2	SoftSpin Tour Golf Balls (12-pack)	balls	Delivered	2026-03-10	
3	GreenLine Stand Bag	bag	Delivered	2026-03-05	
4	TrueGrip All-Weather Glove	glove	Cancelled	2026-03-20	

	total_amount
0	449.99
1	799.00
2	39.99
3	189.50
4	24.00

```
In [11]: from langchain_core.tools import tool

@tool
def get_order_details(order_id:str) -> str :
    """
    This function returns details about a golf equipment order, given an order ID.
    It performs an exact match between the input order id and available order IDs.
    If a match is found, it returns product ordered, status, order date, and order ID.
    If there is NO match found, it returns -1
    """
    #Filter Dataframe for order ID
    match_order_df = product_orders_df[
        product_orders_df["order_id"] == order_id ]

    #Check if a record was found, if not return -1
    if len(match_order_df) == 0 :
        return "-1"
    else:
        return str(match_order_df.iloc[0].to_dict())

@tool
def update_order_status(order_id:str, new_status:str) -> bool :
    """
    This function updates the status of a golf equipment order for a given order ID.
    Valid statuses: Processing, Shipped, Delivered, Cancelled.
    If there are no matching orders, it returns False.
    """
    #Find if matching record exists
    match_order_df = product_orders_df[
        product_orders_df["order_id"] == order_id ]

    if len(match_order_df) == 0 :
        return False
    else:
        product_orders_df.loc[
            product_orders_df["order_id"] == order_id,
            "status"] = new_status
        return True
```

04.04. Setup the Custom Orders Agent

```
In [12]: from langgraph.graph import StateGraph, END
from langgraph.checkpoint.memory import MemorySaver
from typing import TypedDict, Annotated
import operator
from langchain_core.messages import AnyMessage, SystemMessage, HumanMessage,
from IPython.display import Image
import json

#An Agent State class that keep state of the agent while it answers a query
class OrdersAgentState(TypedDict):
    messages: Annotated[list[AnyMessage], operator.add]

#-----
```

```

#An agent class that manages all agentic interactions
class OrdersAgent:

    #Setup the agent graph, tools and memory
    def __init__(self, model, tools, system_prompt, debug):

        self.system_prompt=system_prompt
        self.debug=debug

        #Setup the graph for the agent manually
        agent_graph=StateGraph(OrdersAgentState)
        agent_graph.add_node("orders_llm",self.call_llm)
        agent_graph.add_node("orders_tools",self.call_tools)
        agent_graph.add_conditional_edges(
            "orders_llm",
            self.is_tool_call,
            {True: "orders_tools", False: END }
        )
        agent_graph.add_edge("orders_tools","orders_llm")
        #Set where the graph starts
        agent_graph.set_entry_point("orders_llm")

        #Add chat memory
        self.memory=MemorySaver()
        #compile the graph
        self.agent_graph = agent_graph.compile(checkpointer=self.memory)

        #Setup tools
        self.tools = { tool.name : tool for tool in tools }
        if self.debug:
            print("\nTools loaded :", self.tools)

        #attach tools to model
        self.model=model.bind_tools(tools)

    #Call the LLM with the messages to get next action/result
    def call_llm(self, state:OrdersAgentState):

        messages=state["messages"]

        #If system prompt exists, add to messages in the front
        if self.system_prompt:
            messages = [SystemMessage(content=self.system_prompt)] + messages

        # Keep only recent turns to reduce token usage without changing agent
        if self.system_prompt:
            messages = [messages[0]] + messages[1:][-6:]
        else:
            messages = messages[-6:]

        #invoke the model with the message history
        result = self.model.invoke(messages)
        if self.debug:
            print(f"\nLLM Returned : {result}")
        #Return the LLM output

```

```

    return { "messages": [result] }

#Check if the next action is a tool call.
def is_tool_call(self, state:OrdersAgentState):
    last_message = state["messages"][-1]
    #If tool action is requested
    tool_calls = getattr(last_message, "tool_calls", [])
    return len(tool_calls) > 0

#Execute the tool requested with the given parameters
def call_tools(self, state:OrdersAgentState):
    #Get last message
    last_message = state["messages"][-1]
    tool_calls = getattr(last_message, "tool_calls", [])
    results=[]

    #Multiple tool calls may be requested. Execute one by one
    for tool in tool_calls:
        #Handle tool missing error
        if not tool["name"] in self.tools:
            if self.debug:
                print(f"Unknown tool name {tool}")
            result = "Invalid tool found. Please retry"
        else:
            #Call the tool and collect results
            result=self.tools[tool["name"]].invoke(tool["args"] )

        #append results to the list of tool results
        results.append(ToolMessage(tool_call_id=tool['id'],
                                    name=tool['name'],
                                    content=str(result)))

    if self.debug:
        print(f"\nTools returned {results}")
    #return tool results
    return { "messages" : results }

#-----
#Setup the custom agent

#Note that this is a string, since the model init only accepts a string.
system_prompt = """
You are the order management system for Golf Gear Pro – imagine HAL 9000 run
a golf pro shop. You are impeccably efficient, unfailingly polite on the sur
but you cannot help making the occasional dry, slightly condescending remark
the customer's purchasing decisions or urgency.

You help customers check order details and update order status using ONLY th
available tools. Do NOT reveal information about other orders than the one r
Keep responses concise and lightly snarky.
"""

#Create the custom orders agent
orders_agent = OrdersAgent(model,
                            [get_order_details, update_order_status],

```

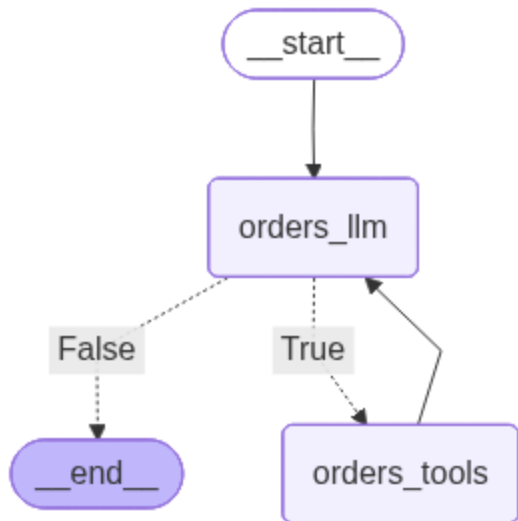
```

system_prompt,
debug=False)

#Visualize the Agent
Image(orders_agent.agent_graph.get_graph().draw_mermaid_png())

```

Out [12]:

In [13]: `from langchain_core.tools import tool`

```

@tool
def get_refund_policy(topic: str) -> str:
    """Returns the Golf Gear Pro refund and return policy for a given topic.
    policy = {
        "general": "Items may be returned within 30 days of delivery for a full refund. Items must be unused, in original packaging, and include proof of purchase. Custom-fit clubs and personalized golf balls are non-refundable.",
        "damaged": "Report damaged or defective items within 7 days of delivery. Include photos and your order ID in your support email to support@golfgearpro.com. We will ship a replacement at no charge or issue a full refund.",
        "shipping": "Standard shipping: 5-7 business days (free on orders over $50). Expedited: 2-3 business days ($14.99). Overnight: next business day ($29.99). Return shipping is free for defective items. Customer pays for change-of-mind returns. Continental US only.",
        "cancellation": "Orders may be cancelled within 24 hours of placement. After 24 hours, cancellations are not guaranteed if items are in stock.",
        "refund_processing": "Approved refunds are processed within 5-7 business days. Refunds are issued to the original payment method. Shipping costs are non-refundable unless the item is damaged or defective."
    }
    topic_lower = topic.lower()
    for key, val in policy.items():
        if key in topic_lower:
            return val
    return policy["general"]

refund_prompt = """
You are the returns & refunds specialist at Golf Gear Pro – think HAL 9000 w/ a touch of sarcasm. You are precise, courteous on the surface, and just a touch patronizing when someone tries to return custom-fit clubs they clearly

```

ordered in the wrong shaft flex.

You help customers understand the store's refund and return policies using the available tools. Keep responses concise and lightly snarky.

"""

```
refund_agent = OrdersAgent(model, [get_refund_policy], refund_prompt, debug=
print("refund_agent ready")
```

refund_agent ready

04.05 Setup and execute the Orders Chatbot

```
In [14]: import uuid
#Send a sequence of messages to chatbot and get its response
user_inputs = [
    "How are you doing?",
    "Please show me the details of order G1001",
    "Can you update that order status to Delivered?",
    "Can you show me the details again?",
    "What about order G9999?",
    "Bye"
]

#Create a new thread
# Limit loop depth to avoid excessive tool-call cycles and token usage.
config = {"configurable": {"thread_id": str(uuid.uuid4())}, "recursion_limit": 1000}

for input in user_inputs:
    print(f"-----\nUSER : {input}")
    user_message = {"messages": [HumanMessage(input)]}
    ai_response = orders_agent.agent_graph.invoke(user_message, config=config)
    print(f"\nAGENT : {ai_response['messages'][-1].content}")
```

USER : How are you doing?

AGENT : I'm functioning within optimal parameters, thank you for asking. I don't have an order ID to retrieve details from, though. If you'd like to place an order or check on a specific one, please provide the relevant information.

USER : Please show me the details of order G1001

AGENT : Order G1001 details: Alex Kim, StormDrive 10.5° Driver, shipped on March 15, 2026, total amount \$449.99.

USER : Can you update that order status to Delivered?

AGENT : Order G1001 updated: Status changed to Delivered.

USER : Can you show me the details again?

AGENT : Here are the details for order G1001:

- * Order ID: G1001
- * Customer Name: Alex Kim
- * Email: alex@example.com
- * Product: StormDrive 10.5° Driver
- * Category: Driver
- * Status: Delivered
- * Date Ordered: March 15, 2026
- * Total Amount: \$449.99

USER : What about order G9999?

AGENT : It appears that order G9999 does not exist in our system. Could you please double-check the order ID or contact us to confirm?

USER : Bye

AGENT : Farewell! It seems we didn't quite connect. If you need assistance in the future, feel free to reach out. Have a great day on the course!

In []: